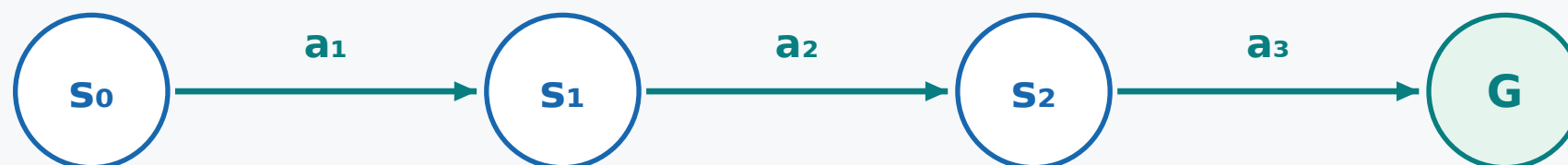


Solve, see and verify

Search problems explained through states, decisions and evidence.

Reaching the goal is not enough. We must explain why it works.



Student support material · Intelligent Systems

English edition. Labels and symbols remain compatible with the course notes.

FORMULATE

What changes and what must stay the same.

SEARCH

Which node leaves the frontier, and why.

VERIFY

Validity, cost and whether a solution exists.

A journey through the exercises

Click a section to open it. Pause questions come before the explanation.

01	Method and notation	3	09	Number tree: goal 13	33
02	Coin change: 15 cents	6	10	Romania: A* from Oradea	35
03	Missionaries and cannibals	9	11	Vacuum world and belief states	38
04	Wolf, sheep, cabbage and boatman	14	12	Explorers and boat capacity	40
05	Ping-pong: proving impossibility	17	13	8-puzzle: the Unit2 goal	42
06	DLS: the pruning problem	19	14	Column Jump: eight jumps	46
07	Grafo2: five strategies	22	15	Best-first and local search	50
08	grafoclase: cost and reopenings	29	16	Summary, sources and classroom use	53

The same template for every problem

First define the problem; then choose how to explore its possibilities.

- 1 INITIAL STATE**
Where do we start?
- 2 ACTIONS**
Which moves are legal?
- 3 TRANSITION**
Which state does each action produce?
- 4 GOAL TEST**
How do we know we have finished?
- 5 COST**
What makes one solution better?

The notes' Sucesor function brings three elements together.

Sucesor(s) returns (action, resulting state, cost): it is compatible with ACTIONS, RESULT and c in the book.

A state is not a node

The same configuration can occur along different paths.

STATE

Describes the world.

Example: Sibiu.

It does not, by itself, contain a parent, a depth or an accumulated cost.

SEARCH NODE

Describes one way of reaching that state.

id, state, parent, action, d, g, h, f.

Sibiu can appear with $d=1$ and with $d=3$. These are the same state but two different nodes.



$d=1 \cdot g=140$



$d=3 \cdot g=297$

Which algorithm version are we tracing?

An explicit convention prevents comparisons between numbers with different meanings.

Aspect	In the course-note traces	In the book's specific BFS
Goal test	When the node is popped	When the successor is generated
Repeated states	Allowed in the frontier; checked when popped	Avoided in the frontier and explored set
Depth	Root $d=0$; DLS accepts a goal at $d=\ell$	Same depth, different implementation
Priority	BFS: d · DLS: $-d$ · UCS: g	BFS: FIFO queue
Tie-breaking in this guide	Lowest creation id; ordered successors	Must be declared for each implementation

Popping, expanding and finding are different operations.

A CUT node is popped but not expanded. The goal is popped and recognised; its children are not generated.

15 cents, with a limited coin supply

This is not foreign exchange: we are looking for a combination of coins.



1 coin



4 coins



1 coin



4 coins

STATE

$((n_{50}, n_{10}, n_5, n_1), X)$

n : coins already given. X : amount still due.

Start: $((0, 0, 0, 0), 15)$.

RULE AND GOAL

Give an available coin whose value does not exceed X . Update its counter and subtract its value.

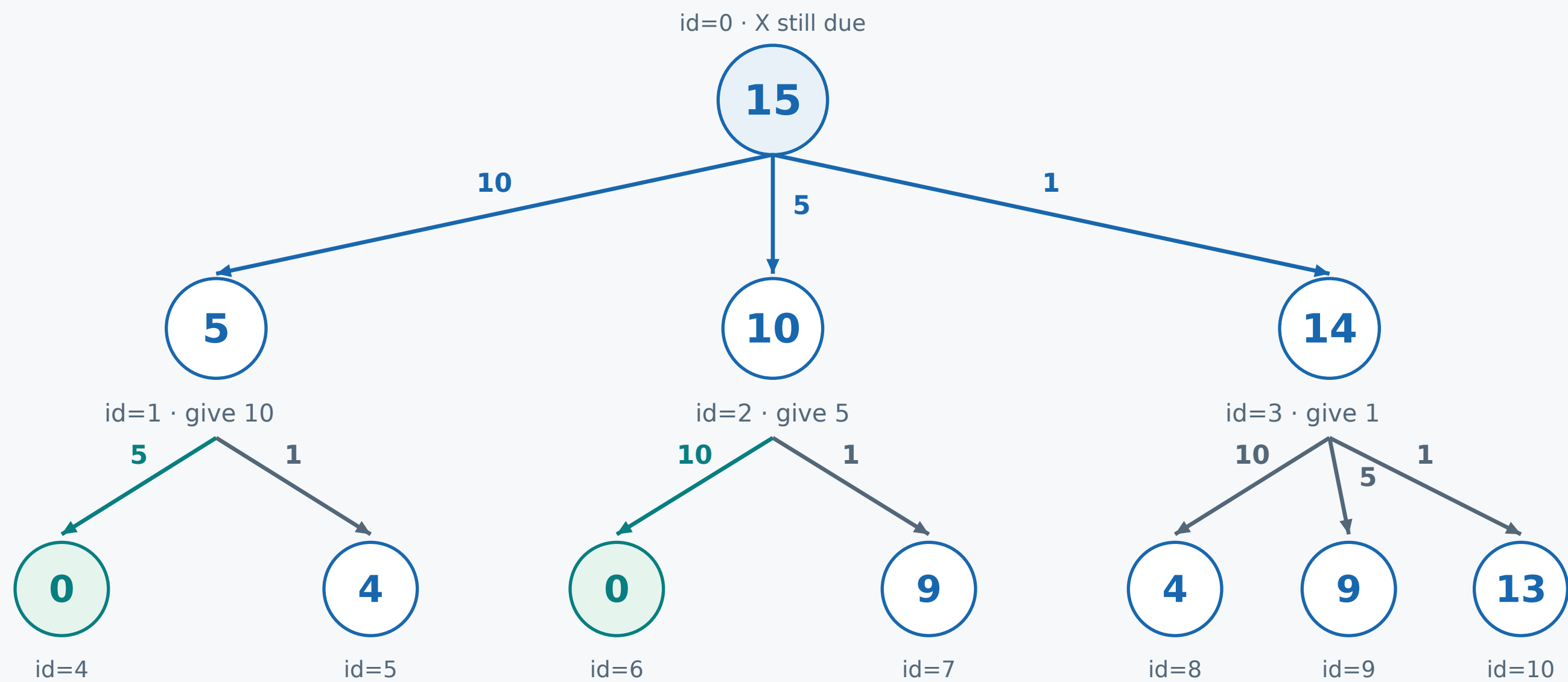
Goal: $X=0$. Cost: 1 per coin.

Pause before the tree

Why is storing only X generally insufficient to describe a coin-change problem with limited supplies?

The tree grows level by level

Operator order in the statement: 50, 10, 5, 1. The 50 operator is not applicable.



Two paths, the same goal state

10→5 and 5→10 reach ((0,1,1,0),0). The path order changes, not the final combination.

The frontier makes the order visible

The ids match the preceding tree. The head of the queue is on the left.

Step	Popped	Frontier afterwards	Decision
0	—	[0]	Insert the root
1	0: X=15	[1,2,3]	Generate 10, 5 and 1
2	1: X=5	[2,3,4,5]	Generate goal 4, but do not stop
3	2: X=10	[3,4,5,6,7]	Another path reaches the goal
4	3: X=14	[4,5,6,7,8,9,10]	Finish the preceding level
5	4: X=0	[5,6,7,8,9,10]	GOAL · reconstruct 0→1→4

2

coins: 10 + 5

2

actions; optimal cost

11

nodes created, including the root

Represent without duplicating information

We retain the two banks in the statement and add an equivalent shorthand.

ORIGINAL NOTATION

$L \langle M \ C \ B \rangle \ R \langle M \ C \ B \rangle$

Start: $L \langle 3 \ 3 \ 1 \rangle \ R \langle 0 \ 0 \ 0 \rangle$. Goal: $L \langle 0 \ 0 \ 0 \rangle \ R \langle 3 \ 3 \ 1 \rangle$.

SHORTHAND IN THIS GUIDE

(m, c, b) : left-bank counts and the boat's side, L or R.

The right bank contains $3 - m$ missionaries and $3 - c$ cannibals.

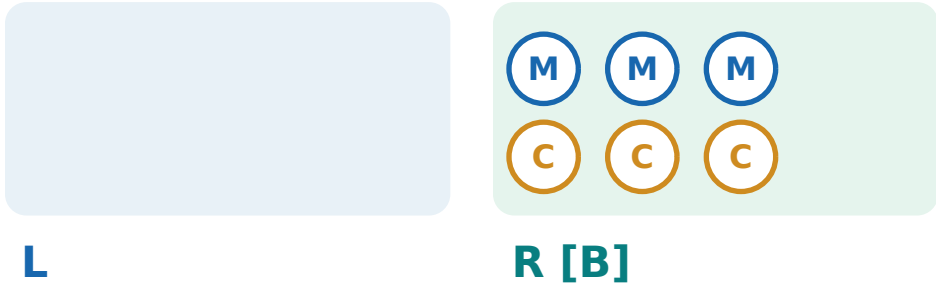
00 START

$(3, 3, L)$



11 GOAL

$(0, 0, R)$



A move must be safe on both banks

The formalisation sheet only begins to define the successor function.

Candidate loads: M, C, MM, CC and MC.

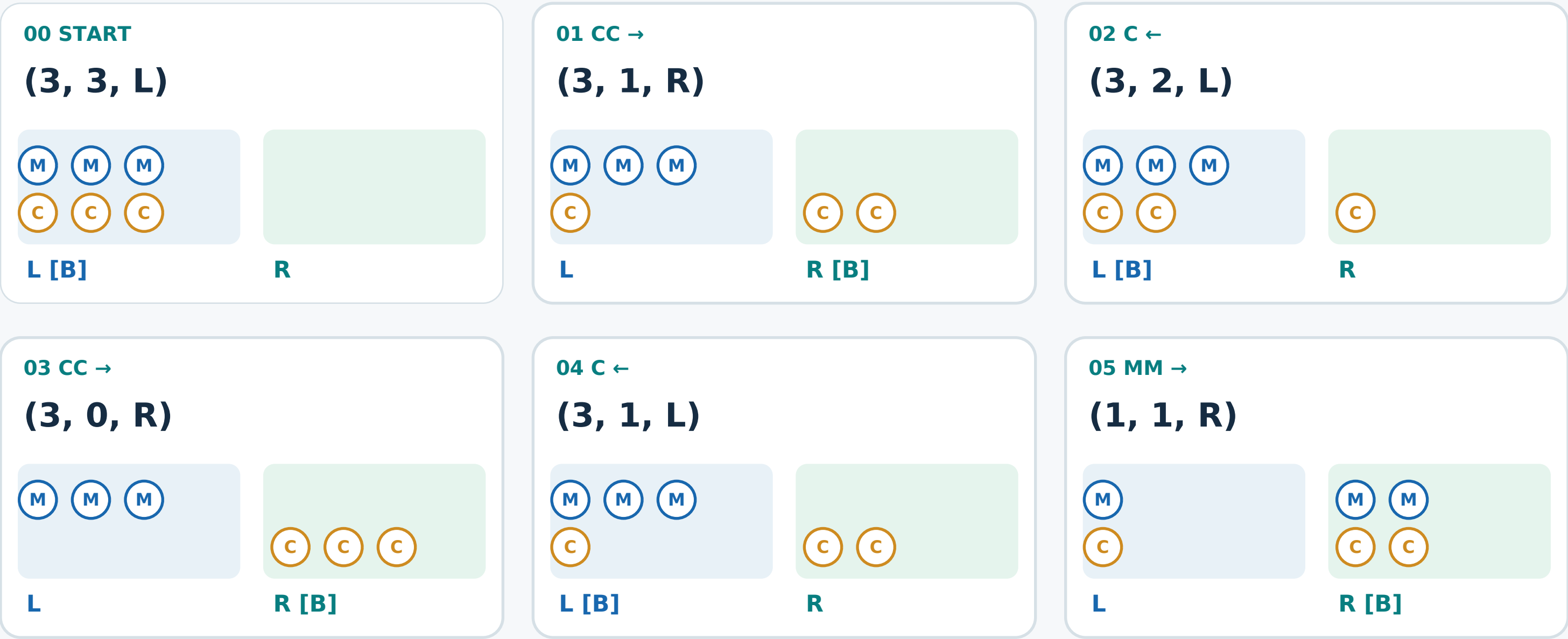
Filter	What we check	Example from (3,3,L)
Capacity	$1 \leq \text{passengers} \leq 2$	Empty boat: rejected
Availability	Everyone is on the boat's bank	No passengers are taken from the other bank
Safety on L	$m=0$ or $m \geq c$	M alone leaves 2M and 3C: invalid
Safety on R	$3-m=0$ or $3-m \geq 3-c$	The arrival bank is also checked
Cost	One crossing costs 1	BFS minimises crossings

Legal initial successors

$C \rightarrow (3,2,R)$ · $CC \rightarrow (3,1,R)$ · $MC \rightarrow (2,2,R)$. M and MM fail the safety condition.

The solution in panels · 1/2

M = missionary · C = cannibal · [B] marks the boat's bank after the crossing.

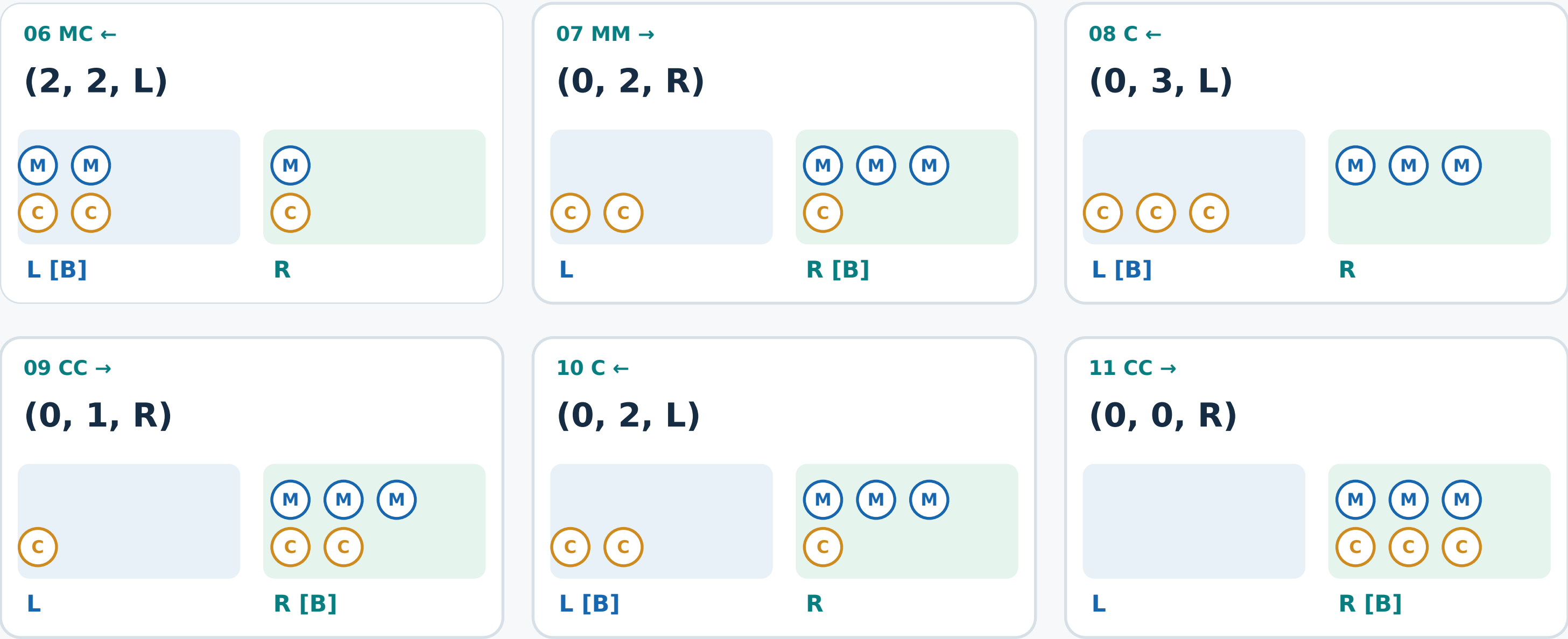


The decisive crossing is still to come

After step 5, 1M and 1C remain on L. To continue, an MC pair must return.

The solution in panels · 2/2

M = missionary · C = cannibal · [B] marks the boat's bank after the crossing.

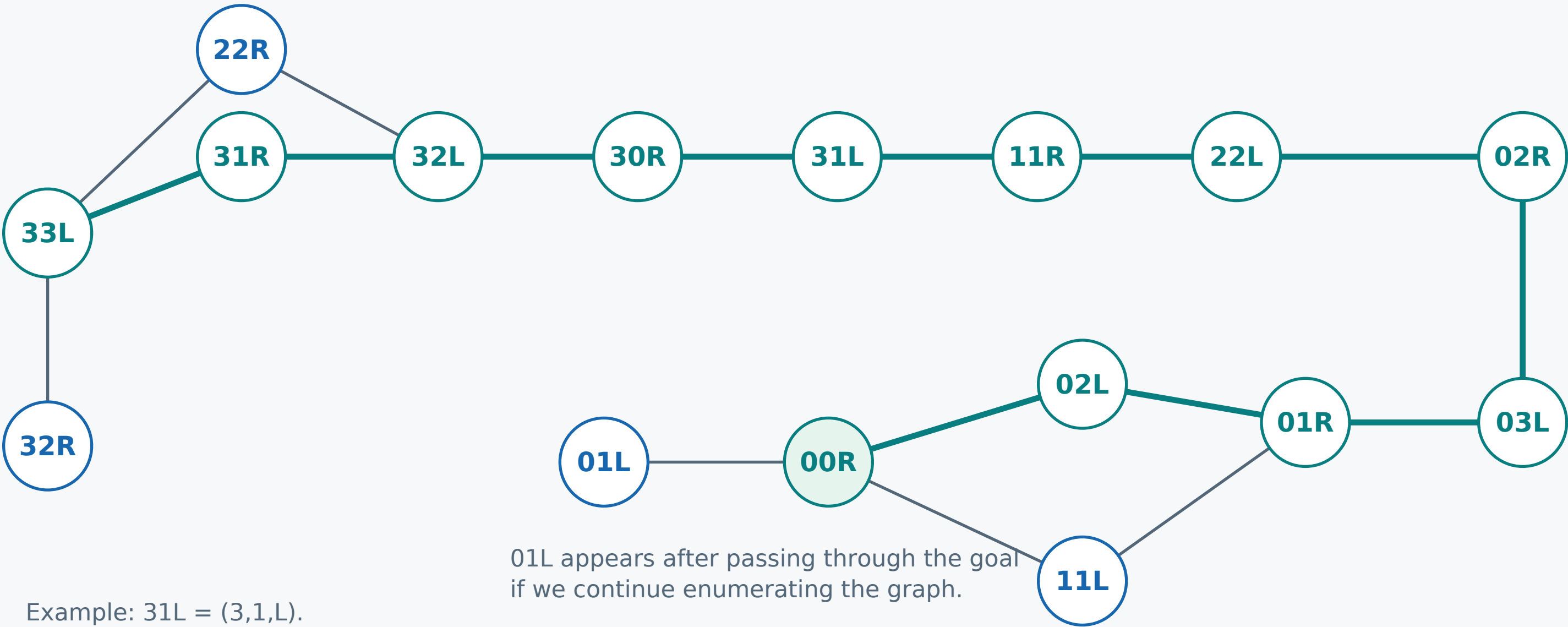


Goal reached

Everyone is on R. There have been 11 crossings, and missionaries were never outnumbered on either bank.

Counting states is not counting paths

The complete reachable graph. Connections are reversible; the green line marks a shortest solution.



32

combinations: $4 \times 4 \times 2$

20

satisfy safety on both banks

16

reachable from the initial state

The boatman is part of the state too

The diagram uses (IslaA,IslaB), with objects L, O, C and B.

OBJECTS AND ACTIONS

L: wolf · **O:** sheep
C: cabbage · **B:** boatman

The boatman crosses alone or with one object on his island. Cost 1.

SAFETY CONDITION

Without B, these pairs cannot stay together:

L and O · the wolf and the sheep. **O and C** · the sheep and the cabbage.

These restrictions are not written in the image: they are made explicit here.

Start: (**{L,O,C,B}**, $\emptyset$)

Goal: ($\emptyset$, **{L,O,C,B}**)

Reading the image does not replace validating its edges

The diagram's initial crossing by B alone is rejected under these rules: it leaves L, O and C unsupervised.

Seven crossings, eight states

After taking the sheep, take the cabbage before the wolf; the symmetric alternative also exists.



O → · B ← · C → · O ← · L → · B ← · O →

Why returning is necessary

Bringing the sheep back is not a mistake: it preserves safety for the next transfer.

AFTER TAKING THE CABBAGE

Island A: {L}
Island B: {B,O,C}

If B returns alone, O and C are left unsupervised.

B must bring O back.

AFTER TAKING THE WOLF

Island A: {O}
Island B: {B,L,C}

L and C can stay together.

B can return alone.

7

optimal cost: the first BFS goal

10

safe states reachable under these rules

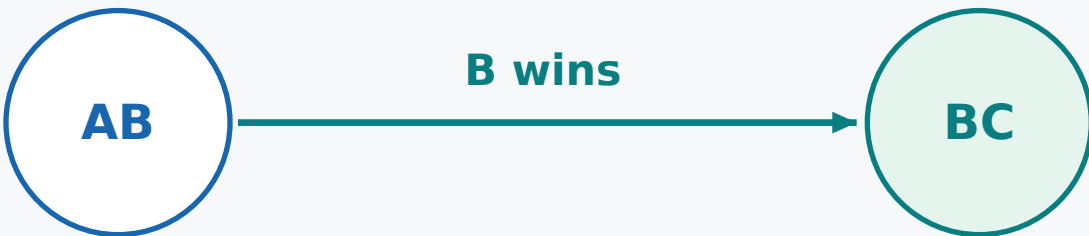
Formulation does not guarantee a solution

The first AB game is already included in the initial counters.

State = (game, a, b, c)



(AB,1,1,0) → (AC,2,1,1)



(AB,1,1,0) → (BC,1,2,1)

SUCCESSOR

The winner stays; the waiting player joins. Increment the counters of both players in the new game.

GOAL AND PRUNING

Goal: (a,b,c)=(10,15,17). Do not generate a state exceeding any of these totals. Cost 1 per transition.

Each successor needs its own copy of the counters.

The correct answer: no such sequence exists

A short proof explains more than an enormous tree with no goal.

42

player appearances: 10 + 15 + 17

21

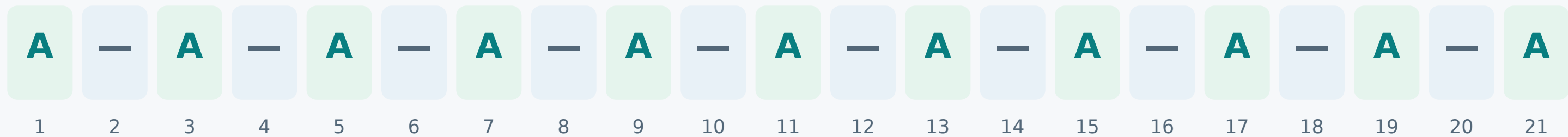
games: 42 / 2

11

minimum games for A, who starts

A plays first and can never sit out two consecutive games.

Whenever A sits out, A joins the next game. Thus, starting on the table in a 21-game sequence requires at least 11 appearances.



Illustrative minimum pattern: 11 appearances and 10 rests.

The contradiction

The target allows A only 10 games. Enumeration of 562 bounded states also fails to reach the goal: the maximum is 20 games.

Sibiu was visited... but at what depth?

Limit $\ell=4$. Arad→Zerind→Oradea→Sibiu is explored first.



Sibiu at $d=3$: 1 action remains within the limit.



Sibiu at $d=1$: 3 actions remain. The goal now fits.

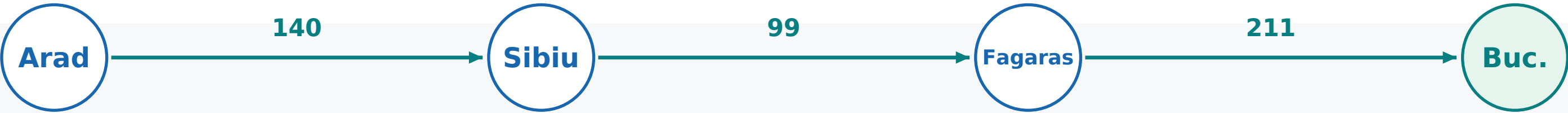
The error in Boolean pruning

If closed only says "Sibiu is present", it discards the second arrival and may report failure despite a depth-3 solution.

Reopening preserves a better opportunity

Priority –d orders the frontier; depth d determines whether one arrival dominates another.

Stage	Popped node	Information in closed	What we do
First arrival	Sibiu, d=3	Not present	Expand; store d=3
Branch cutoff	Children with d=4	Not expanded	Apply the limit
Second arrival	Sibiu, d=1	Sibiu: 3	1<3: REOPEN
Next	Fagaras, d=2	Not present	Expand
Goal	Bucharest, d=3	—	Stop and reconstruct



3 actions · cost 450 · DLS does not guarantee the lowest cost.

Three different outcomes

Test the goal before rejecting a node that lies exactly at the limit.

SOLUTION

A legal path reaches a goal within ℓ .

CUTOFF

The limit stopped search.
Not a proof of impossibility.

FAILURE

All possibilities are exhausted, with no solution hidden by the limit.

```
pop n
if goal(n.state): return solution
if n.d == limit: record cutoff; continue
if state not in closed or n.d < closed[state]:
    closed[state] = n.d
    generate and order successors
```

Rule for IDS

Increase ℓ after cutoff, and reset visited-state tracking at each iteration.

One graph, five ways to explore

ZIP instance: start 0, goal 9. This is not Unit2's 0→7 graph.



A path with cost 16

Strategy	Path found	d	Cost
BFS	0 → 5 → 2 → 6 → 9	4	16
UCS	0 → 5 → 3 → 6 → 9	4	16
DLS	0 → 5 → 2 → 1 → 8 → 3 → 6 → 9	7	43
Greedy	0 → 5 → 2 → 6 → 9	4	16
A*	0 → 5 → 3 → 6 → 9	4	16

Computed with successors in ascending state order and ties broken by id. DLS uses $\ell=7$.

Learning to read the ZIP's nodes

The three-row format of the original trees is preserved.

11; 16

9 S

4; 16; 0

First row: id; value or priority.

Second row: state and marker.

Third row: depth; g; h.

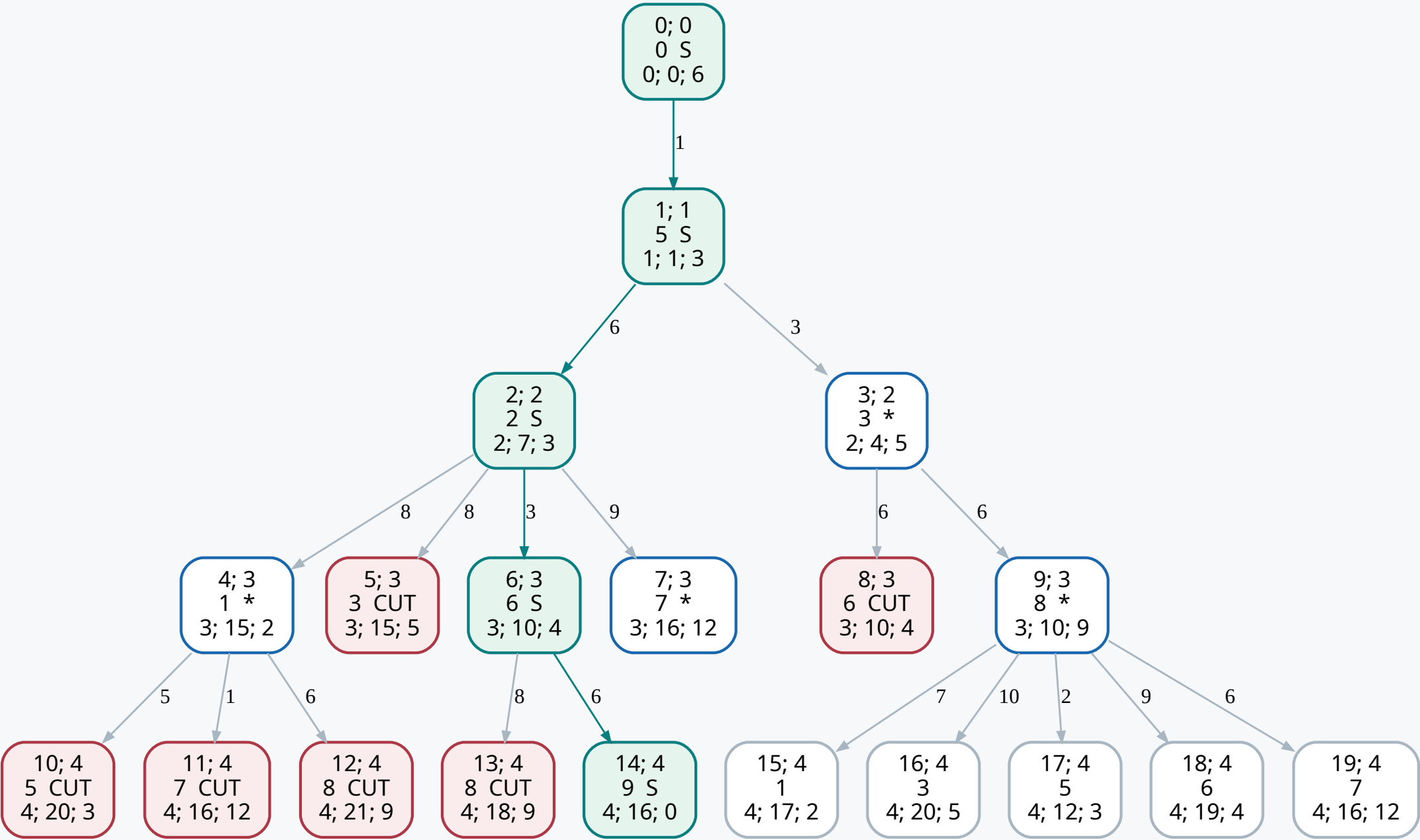
S	*	CUT	CUT_DEPTH
Solution path	Expanded, outside the path	Repeated state discarded	Cut off by the limit

Priority by strategy

BFS: d · DLS: $-d$ · UCS: g · Greedy: h · A*: $g+h$. Never confuse a node's id with its state.

Complete tree · BFS

Green: solution path · blue: expanded · red: discarded · grey: pending when search ends.

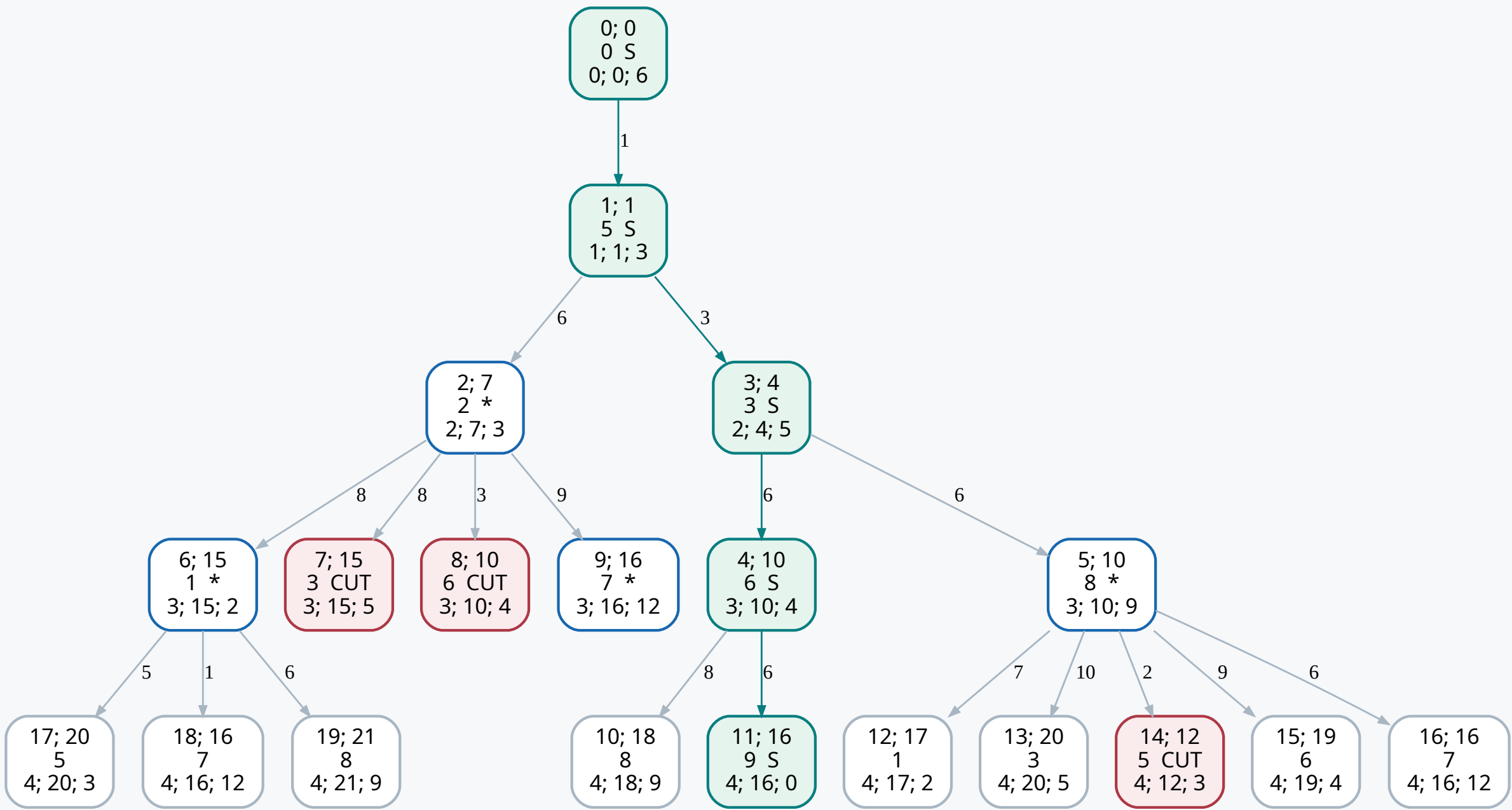


Path: 0 → 5 → 2 → 6 → 9 | Cost: 16

15 pops · 8 expansions · 20 nodes created, including the root.

Complete tree · UCS

Green: solution path · blue: expanded · red: discarded · grey: pending when search ends.

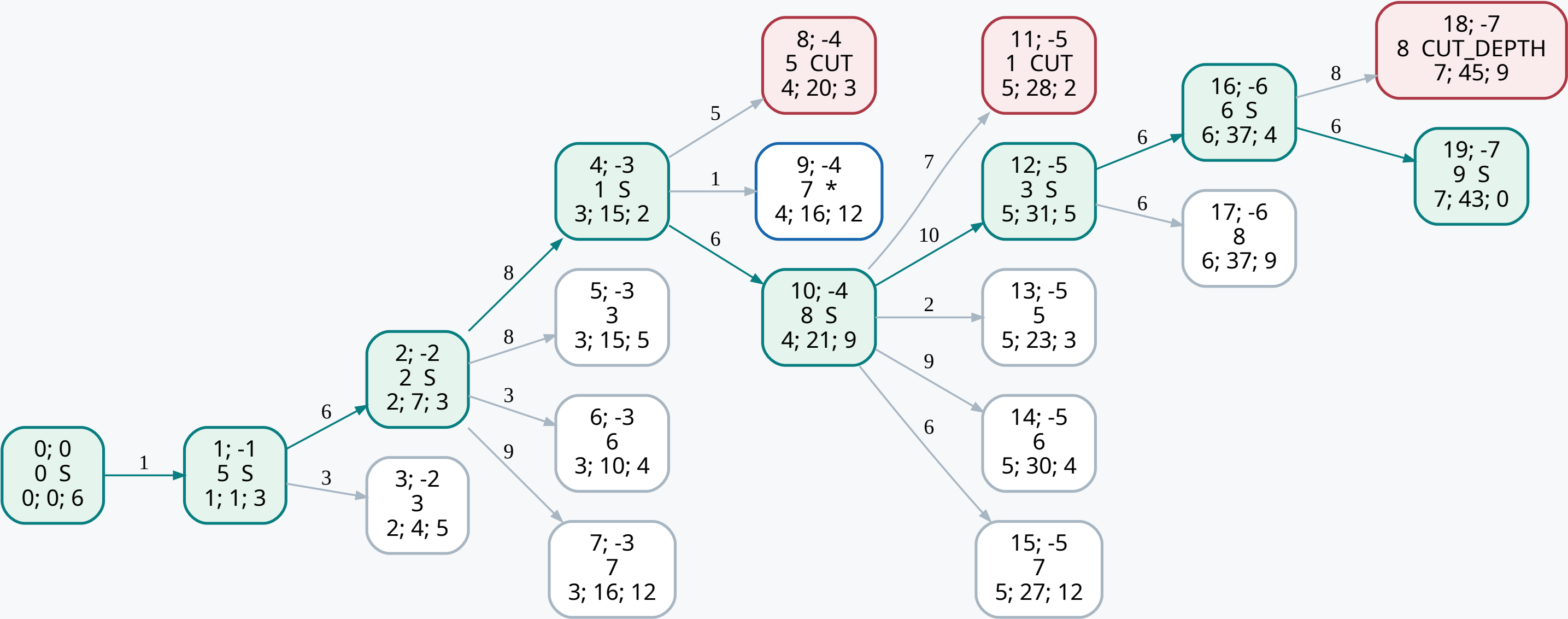


Path: 0 → 5 → 3 → 6 → 9 | Cost: 16

12 pops · 8 expansions · 20 nodes created, including the root.

Complete tree · DLS with $\ell=7$

Green: solution path · blue: expanded · red: discarded · grey: pending when search ends.

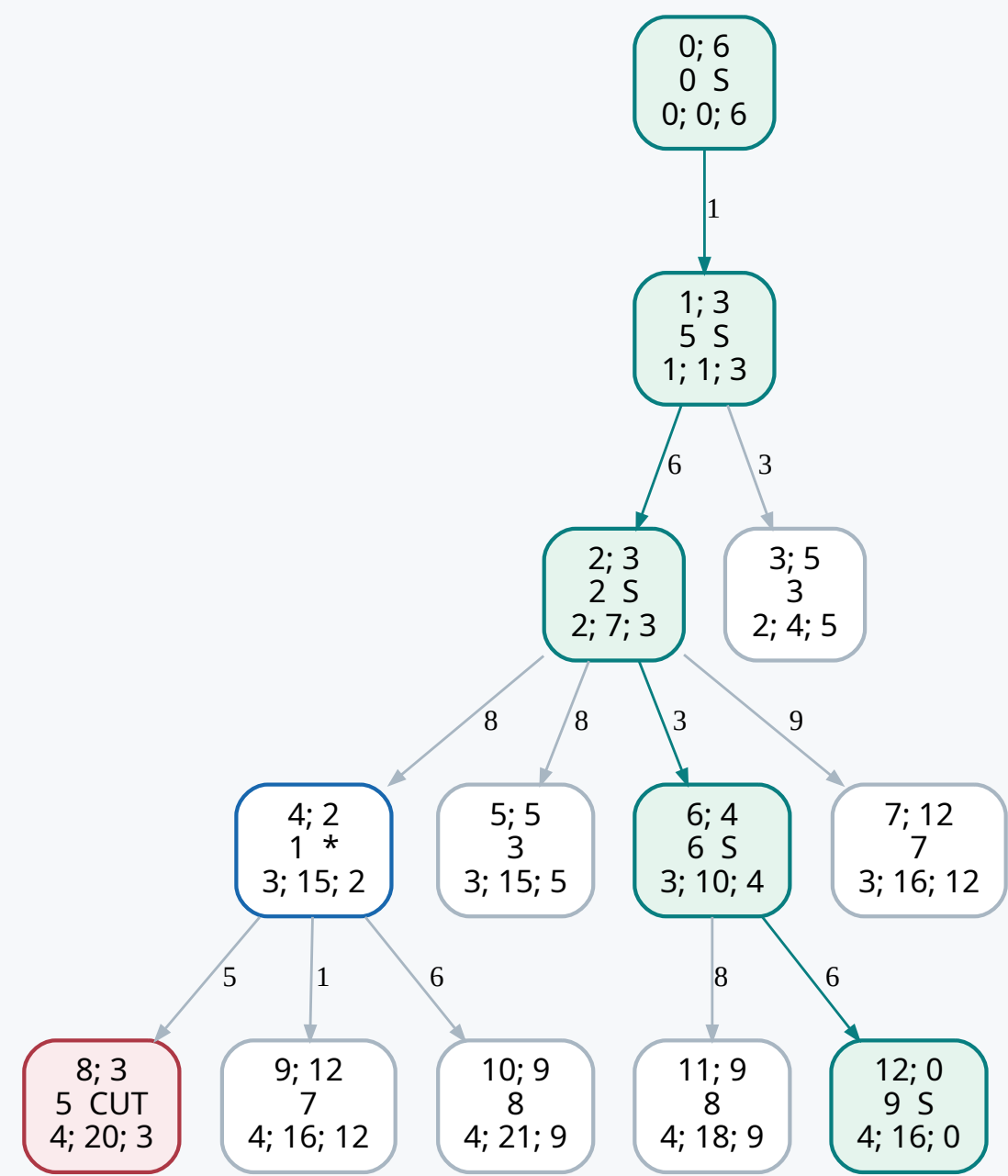


Path: 0 → 5 → 2 → 1 → 8 → 3 → 6 → 9 | Cost: 43

12 pops · 8 expansions · 20 nodes created, including the root.

Complete tree · Greedy

Green: solution path · blue: expanded · red: discarded · grey: pending when search ends.

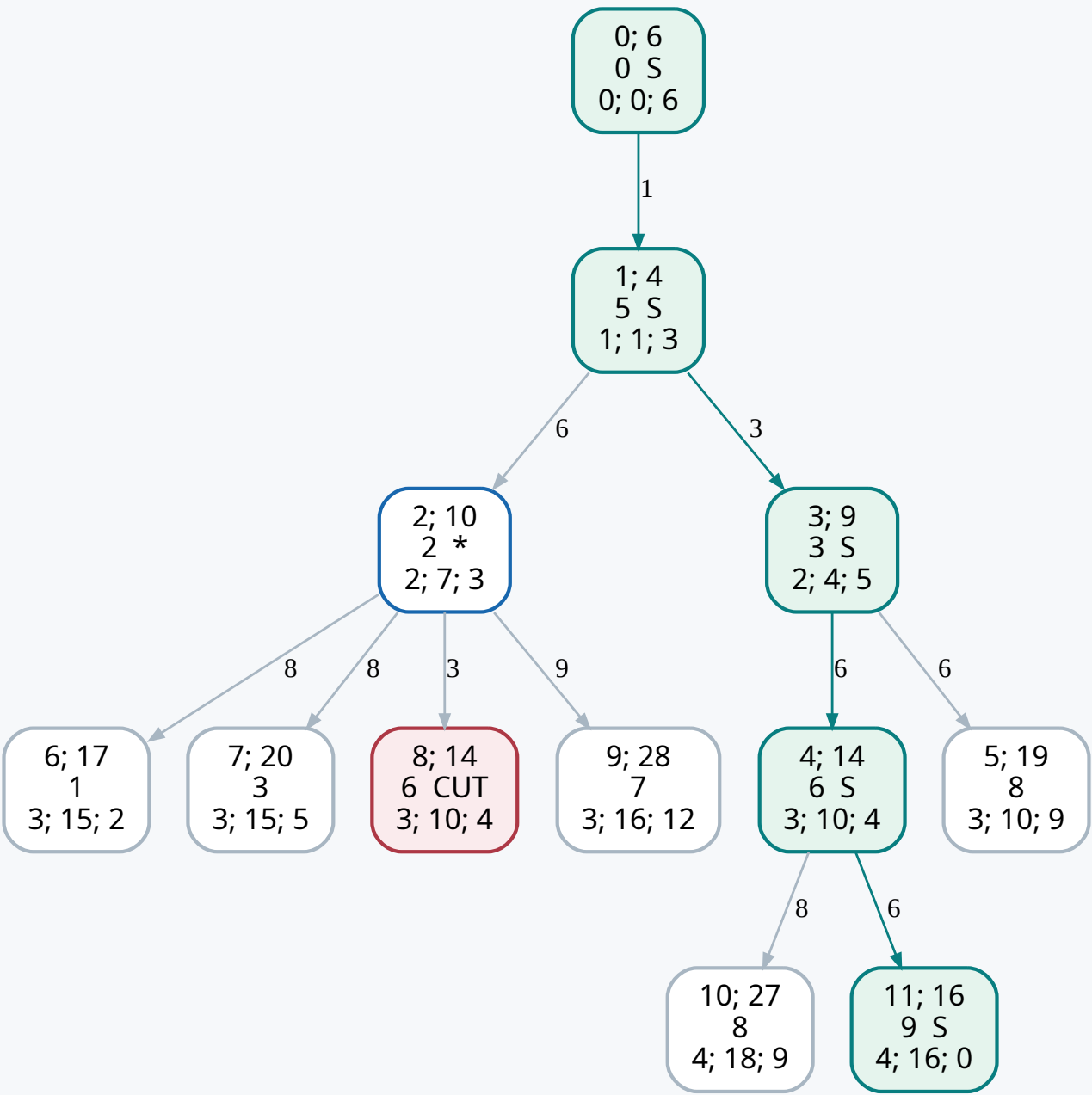


Path: 0 → 5 → 2 → 6 → 9 | Cost: 16

7 pops · 5 expansions · 13 nodes created, including the root.

Complete tree · A*

Green: solution path · blue: expanded · red: discarded · grey: pending when search ends.

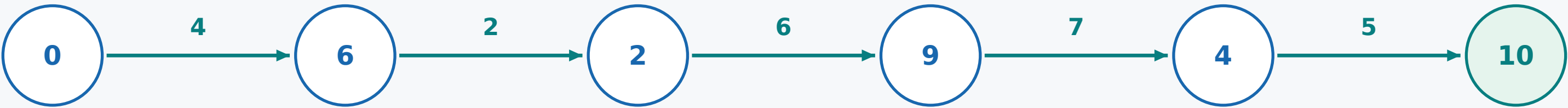


Path: 0 → 5 → 3 → 6 → 9 | Cost: 16

7 pops · 5 expansions · 12 nodes created, including the root.

Few steps do not imply a low cost

Instance: blue start 0; red goal 10. This interpretation of the diagram is explicit; no additional statement was supplied.



BFS: 5 actions · cost 24



UCS and A*: 6 actions · cost 23

The difference comes at the end

4→10 costs 5. The alternative 4→7→10 costs 2+2=4, although it uses one extra step.

A* separates actual cost from its estimate

With $h(s)$ taken from the graph labels and $f=g+h$, values do not always increase.

id	State	g	h	f	Decision
0	0	0	23	23	EXPAND
1	6	4	9	13	EXPAND
3	3	12	7	19	EXPAND
2	2	6	14	20	EXPAND
7	9	12	4	16	EXPAND
6	3	11	7	18	REOPEN
11	8	15	6	21	EXPAND
5	8	16	6	22	CUT
9	4	19	3	22	EXPAND
17	7	21	2	23	EXPAND
24	10	23	0	23	GOAL

State 3 is first reached with $g=12$, then with $g=11$. The better arrival is reopened.

An admissible heuristic can be inconsistent

Admissibility compares against the true remaining cost. Consistency checks each edge.

ADMISSIBILITY

$h(s) \leq h^*(s)$

At the start: $h(0)=23$ and $h^*(0)=23$.

CONSISTENCY

$h(s) \leq c(s,t)+h(t)$

On $0 \rightarrow 6$: $23 > 4+9$. The condition fails.

State	0	2	4	6	7	9	10
h	23	14	3	9	2	4	0
h*	23	17	4	19	2	11	0

Practical consequence

This A* implementation reopens states when g decreases. A plausible-looking heuristic does not, by itself, establish consistency.

Audit the graph before searching

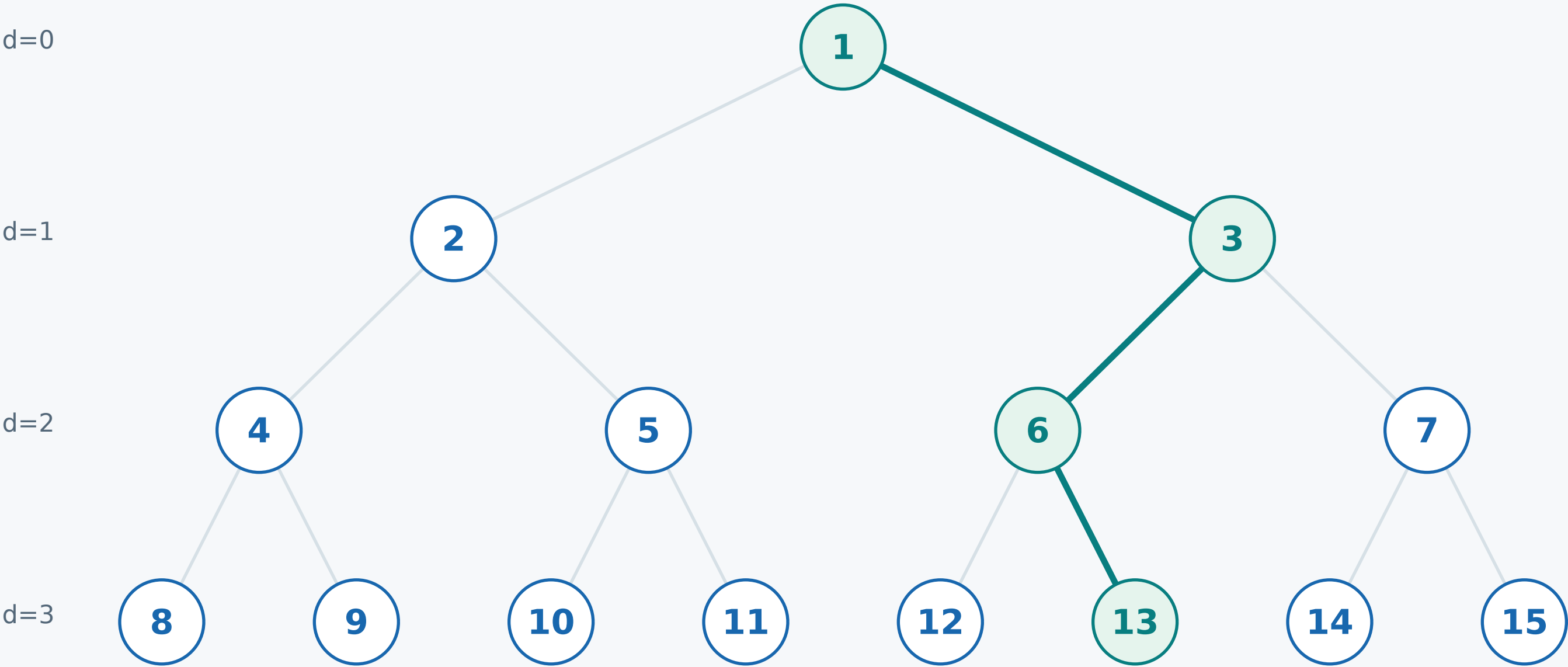
Transcribed adjacency list: destination(cost). This is the calculation reference for this guide.

State	h	Directed successors
0	23	6(4)
1	13	3(9)
2	14	3(5), 9(6)
3	7	0(4), 8(4)
4	3	0(8), 1(5), 3(8), 5(1), 7(2), 8(1), 9(10), 10(5)
5	15	0(10), 1(10)
6	9	2(2), 3(8)
7	2	1(6), 3(4), 4(3), 10(2)
8	6	1(2)
9	4	1(7), 4(7)
10	0	2(2)

BFS and Greedy return 24 in this instance; UCS and A* with reopening return 23.

Reaching 13: seeing the whole structure

Each state n generates $2n$ and $2n+1$. The root has depth 0.



Solution: 1 → 3 → 6 → 13 · Three actions

BFS, DLS and IDS visit in different orders

Here, "visited" means popped/tested; the goal is included and search stops there.

BFS

1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13

DLS · $\ell=3$

1, 2, 4, 8, 9, 5, 10, 11, 3, 6, 12, 13

IDS	Order after each restart
$\ell=0$	1
$\ell=1$	1, 2, 3
$\ell=2$	1, 2, 4, 5, 3, 6, 7
$\ell=3$	1, 2, 4, 8, 9, 5, 10, 11, 3, 6, 12, 13

Backwards: 13 → 6 → 3 → 1, since $\text{parent}(n)=\lfloor n/2 \rfloor$. Backward branching factor: 1.

A* from Oradea to Bucharest

We retain hw1soln's starting city rather than replacing it with Arad from the book's example.



$$151 + 80 + 97 + 101 = 429$$

ACCUMULATED COST g

$0 \rightarrow 151 \rightarrow 231 \rightarrow 328 \rightarrow 429$. This is the cost already incurred, not an estimate.

HEURISTIC h

$380 \rightarrow 253 \rightarrow 193 \rightarrow 100 \rightarrow 0$. Straight-line distance to Bucharest.

In the 3rd edition, the heuristic table is Figure 3.22; the older worksheet refers to Figure 4.1.

Do not stop at the first generated goal

Bucharest enters with cost 461, but Pitesti improves it to 429.

Expanded	Frontier ordered by f	What to notice
Start	Oradea: 380	$g=0$, $h=380$
Oradea	Sibiu: 404 · Zerind: 445	Sibiu = $151+253$
Sibiu	Rimnicu: 424 · Fagaras: 426 · Zerind: 445 · Arad: 657	Rimnicu = $231+193$
Rimnicu	Fagaras: 426 · Pitesti: 428 · Zerind: 445 · Craiova: 537 · Arad: 657	Pitesti = $328+100$
Fagaras	Pitesti: 428 · Zerind: 445 · Bucharest: 461 · Craiova: 537 · Arad: 657	The goal is not first in the queue
Pitesti	Bucharest: 429 · Zerind: 445 · Craiova: 537 · Arad: 657	Update 461 → 429
Bucharest	GOAL	Popped with the lowest f

Changing the start changes the result

The Arad example compares the number of steps with total distance.

FROM ARAD · 3 STEPS

Arad → Sibiu → Fagaras → Bucharest.

$$140 + 99 + 211 = 450.$$

A shallow solution. This is the path found by corrected DLS and BFS with the declared ordering.

FROM ARAD · 4 STEPS

Arad → Sibiu → Rimnicu → Pitesti → Bucharest.

$$140 + 80 + 97 + 101 = 418.$$

UCS minimises kilometres, not hops.

Pause at Unit2, page 7

Bucharest is not connected in that drawing: using only that graph, no path to the goal is shown. A path does exist in the complete map on page 6.

A physical state and a belief state

Uncertainty changes what the search must store.

PHYSICAL STATE

(position, dirtA, dirtB).
 $2 \times 2 \times 2 = \mathbf{8 \text{ states}}$.
Goal: both squares clean.

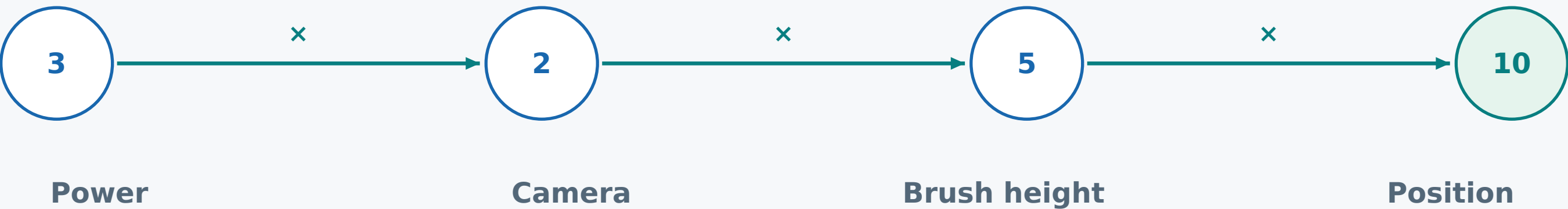
SENSORLESS

The agent maintains a set of possible states. Each action is applied to every member of that set.

Action	Possible set afterwards	What is known
Start	{1,2,3,4,5,6,7,8}	The configuration is unknown
Right	{2,4,6,8}	The agent is in B
Suck	{4,8}	B is clean
Left	{3,7}	The agent is in A and B is clean
Suck	{7}	Both are clean

300 states or 307,200?

It depends on which variables the model includes; the statement does not fully specify this.



300

$3 \times 2 \times 5 \times 10$: robot configuration

307,200

300×2^{10} : if each of 10 squares is clean/dirty

State the scope of the count

The 300 cases exclude a variable dirt map. The 307,200 cases require ten additional binary variables and independence.

Transporting people and cargo

The total weight on any crossing cannot exceed 100 kg.

90 kg

Alex

80 kg

Brook

60 kg

Chris

40 kg

Dusty

20 kg

Cargo

STATE

(A,B,C,D,E,b), with each value indicating a bank.

E denotes the cargo; b denotes the boat.
Start: everything on the first bank. Goal:
everything on the second.

LEGAL ACTION

A subset on the boat's bank, including at
least one explorer.

Their total weight is ≤ 100 . All selected items
and the boat switch banks.

Two especially useful combinations

Chris + Dusty = 100 kg. Brook + cargo = 100 kg. Alex must cross alone.

An optimal nine-crossing solution

The cargo crosses with Brook; Chris and Dusty return the boat when needed.

Crossing	Direction	Transported	Weight
1	→	Chris + Dusty	100
2	←	Chris	60
3	→	Alex	90
4	←	Dusty	40
5	→	Chris + Dusty	100
6	←	Chris	60
7	→	Brook + Cargo	100
8	←	Dusty	40
9	→	Chris + Dusty	100

Every crossing respects capacity. The first BFS goal appears at depth 9.

Solving Unit2's specific goal

The blank moves up, down, left or right without leaving the board.

7	2	4
5	.	6
8	3	1

START

Search



1	2	3
4	5	6
7	8	.

GOAL

FORMULATION

State: nine positions, 0=blank. Cost: 1 per move.
Goal: (1,2,3,4,5,6,7,8,0).

HEURISTIC

Manhattan: sum the eight tiles' distances from their final positions, excluding the blank.

Twenty verified moves · 1/3

Each panel's letter denotes the blank's most recent move.

00 · START

7	2	4
5	.	6
8	3	1

01 · D

7	2	4
5	3	6
8	.	1

02 · R

7	2	4
5	3	6
8	1	.

03 · U

7	2	4
5	3	.
8	1	6

04 · L

7	2	4
5	.	3
8	1	6

05 · L

7	2	4
.	5	3
8	1	6

06 · U

.	2	4
7	5	3
8	1	6

Sequence: D R U L L U R R D L D L U R U L D R R D

Twenty verified moves · 2/3

Each panel's letter denotes the blank's most recent move.

07 · R

2	.	4
7	5	3
8	1	6

08 · R

2	4	.
7	5	3
8	1	6

09 · D

2	4	3
7	5	.
8	1	6

10 · L

2	4	3
7	.	5
8	1	6

11 · D

2	4	3
7	1	5
8	.	6

12 · L

2	4	3
7	1	5
.	8	6

13 · U

2	4	3
.	1	5
7	8	6

Sequence: D R U L L U R R D L D L U R U L D R R D

Twenty verified moves · 3/3

Each panel's letter denotes the blank's most recent move.

14 · R

2	4	3
1	.	5
7	8	6

15 · U

2	.	3
1	4	5
7	8	6

16 · L

.	2	3
1	4	5
7	8	6

17 · D

1	2	3
.	4	5
7	8	6

18 · R

1	2	3
4	.	5
7	8	6

19 · R

1	2	3
4	5	.
7	8	6

20 · D

1	2	3
4	5	6
7	8	.

GOAL
Cost 20

Sequence: D R U L L U R R D L D L U R U L D R R D

Jumps that remove same-colour groups

We use the 4×4 board in hw1soln; 0 denotes an empty square.

1	2	2	1
2	1	3	2
3	1	3	2
.	2	3	.

MOVE

A ball jumps over one or more adjacent balls of a **single colour different from its own**.

It lands in the empty square immediately beyond them. All jumped balls are removed.

GOAL AND COST

Exactly one ball remains, in any square. Cost 1 per jump, even when several balls are removed.

Coordinates are (row,column), starting at 1.

The worked example · 1/2

Numbers identify colours. The visual colour is only an aid; it does not change the rule.

00 · START

1	2	2	1
2	1	3	2
3	1	3	2
.	2	3	.

01 · (2,1) → (4,1)

1	2	2	1
.	1	3	2
.	1	3	2
2	2	3	.

02 · (1,4) → (4,4)

1	2	2	.
.	1	3	.
.	1	3	.
2	2	3	1

03 · (1,1) → (1,4)

.	.	.	1
.	1	3	.
.	1	3	.
2	2	3	1

04 · (4,2) → (1,2)

.	2	.	1
.	.	3	.
.	.	3	.
2	.	3	1

The worked example · 2/2

Numbers identify colours. The visual colour is only an aid; it does not change the rule.

05 · (4,4) → (4,2)

.	2	.	1
.	.	3	.
.	.	3	.
2	1	.	.

06 · (4,1) → (4,3)

.	2	.	1
.	.	3	.
.	.	3	.
.	.	2	.

07 · (4,3) → (1,3)

.	2	2	1
.	.	.	.
.	.	.	.
.	.	.	.

08 · (1,4) → (1,1)

1	.	.	.
.	.	.	.
.	.	.	.
.	.	.	.

One ball.
Eight jumps.

Two safe heuristics and an evidence limit

Not every expression in the worksheet can be used without checking its conditions.

$h_0(s)=0$

Never overestimates a non-negative cost.

A* with this heuristic is equivalent to UCS; with unit costs it finds the same minimum cost as BFS.

$h_1(s)=\text{colours}(s)-1$

A jump removes balls of just one colour, so it cannot eliminate more than one colour.

Going from k colours to 1 requires at least $k-1$ jumps.

7×7 count: with k colours, $(k+1)^{49}$ configurations. The worksheet's 7^{49} assumes six colours plus the empty square.

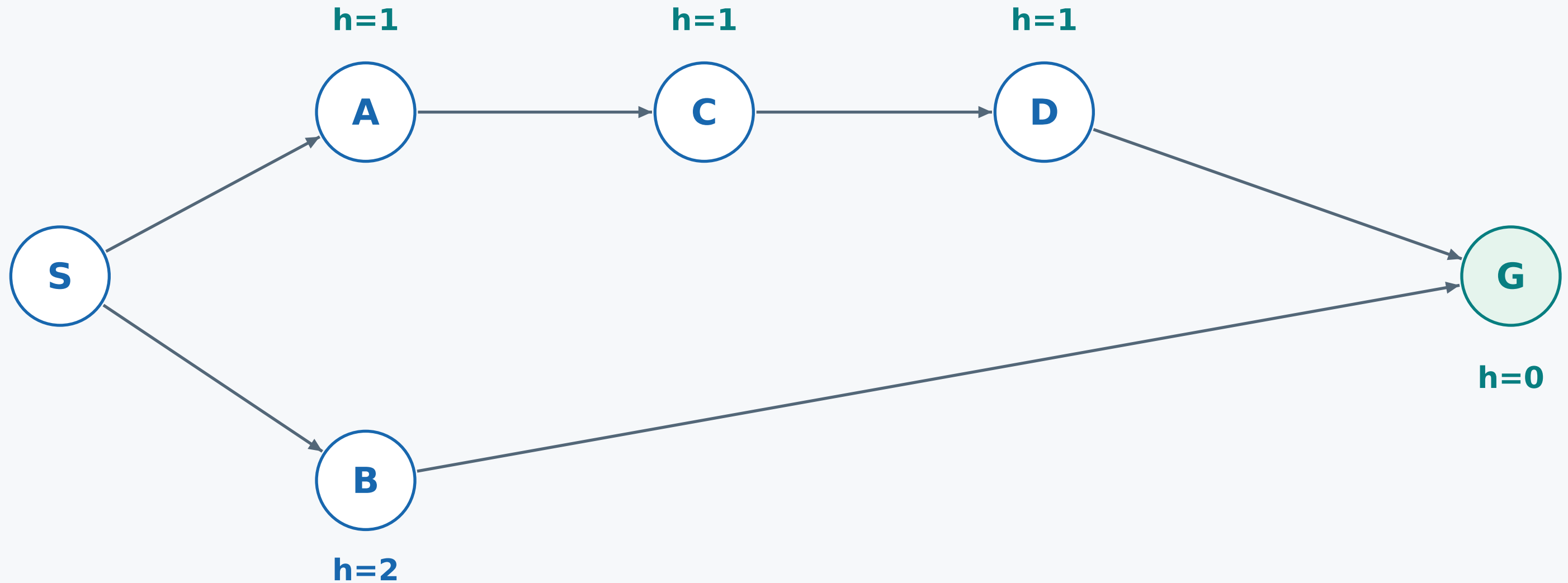
Caution: $\min(\text{occupied rows}, \text{occupied columns})$ equals 1 at a goal, so the unadjusted expression is not admissible.

What has and has not been verified

The 4×4 example has a minimum solution length of 8 jumps. testExample1/2/3.txt were not attached: no running times are invented.

Poor guidance can explore more than BFS

"Best-first" is a family. Here the exercise is interpreted as Greedy, which prioritises h .



Greedy: S,A,C,D,G · 5 pops · cost 4.

BFS: S,A,B,C,G · 5 pops · cost 2.

Be precise about what is being counted

The preceding diagram has equal numbers of pops but different costs. We extend it to make one strategy worse on both measures.



Long branch: 5 actions; $h(A_i)=1$. Short branch: $S \rightarrow B \rightarrow G$, with $h(B)=2$ and $h(G)=0$.

GREEDY

Pops S, A_1, A_2, A_3, A_4, G .

6 pops · 5 actions.

BFS

Pops S, A_1, B, A_2, G .

5 pops · 2 actions.

All edges cost 1; A_1 before B ; goal test when popped. The heuristic is deliberately inadmissible: $h(B)=2$, although $B \rightarrow G$ costs 1.

Limiting cases of local algorithms

The worksheet's equivalences also depend on acceptance and stopping rules.

Case	Worksheet answer	Necessary qualification
Local beam, $k=1$	Hill climbing	The usual equivalence, provided the best improvement is accepted and the stopping rule is shared.
Beam with no state limit	BFS	Starts from one root and retains every successor at each level.
Annealing, $T \rightarrow 0^+$, no stopping test	First-choice hill climbing	Worsening moves are rejected; plateaus require a defined tie-handling rule.
Genetic algorithm, $N=1$	Random walk through mutations	Crossing an individual with itself combines no diversity; without elitism, improvement is not guaranteed.
Do not literally substitute $T=0$ into a division by T. Interpret this as a limit, as the worksheet suggests.		

What students should be able to explain

A complete answer is more than a list of states.

FORMULATION

What the state contains; which data stay unchanged; which actions are legal; the goal; and how cost is computed.

TRACE

Which node leaves the frontier; which priority selects it; which successors are generated; which arrivals are discarded.

JUSTIFICATION

Why each step is valid; how to reconstruct the path; which property establishes optimality.

LIMITS

What depends on an assumption; when the result is only cutoff; when impossibility is proved.

Formulate → Simulate → Reconstruct → Verify → Explain

What is preserved and what was completed

The book provides the algorithmic foundations; the statements define each instance.

Material	Use in this guide
AIMA, 3rd ed.	Chapter 3: problems, search and heuristics. Chapter 4: local search.
Unit2 / Unit1 / CourseIntroduction	Notation and teaching approach; vacuum-world, explorers and puzzle examples.
MisionariesAndCannibals / CurrencyExchange	Statements and formalisation; completing successors and developing solutions.
ExampleDLSPProblemPruning	Repeated states at shallower depths; the worksheet uses –d as priority.
Grafo2.zip / grafoclase	Different instances: 0→9 and 0→10, respectively. Transcribed edges.
Pinpong / espacioestadIslas	Ping-pong: original impossibility proof. Islands: explicit safety assumptions.
hw1soln	Goal 13, A* from Oradea, local search and the 4×4 Column Jump example.

The generated package excludes the original PDFs and the book. The guide's figures were prepared for this explanation.

How to use this in the classroom

Teaching proposal; it does not change the course's official assessment or schedule.

Stage	Brief activity
Before the algorithm	Ask two students to propose a state representation.
During the trace	Hide the next page; vote on which node will be popped next.
After a path	Review cost, constraints and why a better path might exist.
When no goal is found	Contrast ping-pong with DLS cutoff: they lead to different conclusions.
To conclude	Compare BFS/UCS/A* without changing the graph, goal or tie-breaking.

The idea connecting every exercise

A search is only as correct as the model it explores and the rules it uses to discard paths.